

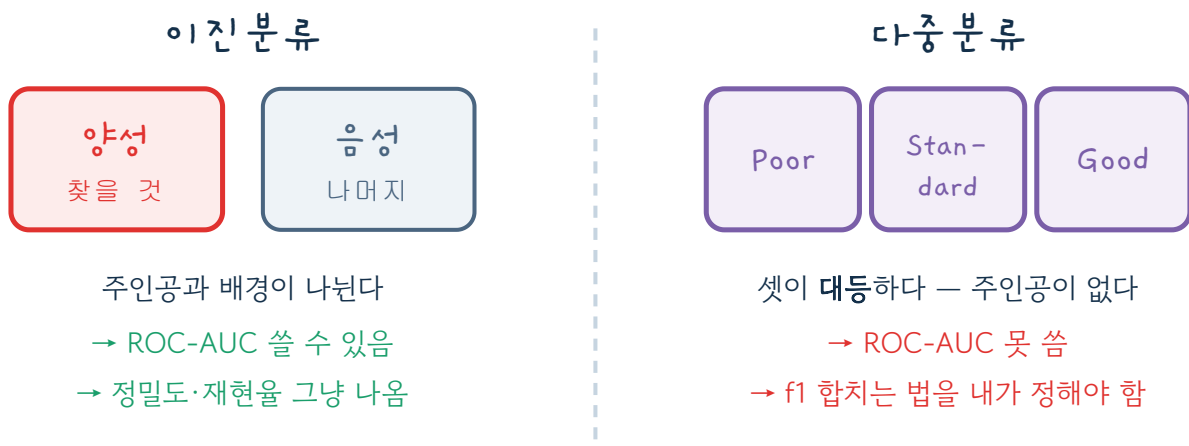
다중분류

답이 셋 이상인 문제.

신용등급 Poor / Standard / Good 처럼요.

Q1 이진분류와 뭐가 다를까

겉보기엔 "선택지가 늘었다"뿐입니다. 그런데 그 하나 때문에 채점 방식이 통째로 바뀝니다.



"누가 양성이야?"라는 질문이 사라지는 순간, 채점표도 같이 바뀝니다.

ROC-AUC는 "양성 vs 음성" 구도에서만 계산되는 점수입니다. 등급 세 개가 대등하면 그 구도 자체가 없으니 쓸 수 없어요.

Q2 문제정의 — 정할 것 네 가지

신용등급 자동 분류를 예로 들어볼게요.

#	질문	답
1	무엇을 맞히나	Credit_Score (신용등급)
2	답이 몇 종류인가	3종류 → 다중분류
3	양성(1)은 무엇인가	해당 없음 — 세 등급이 대등함
4	어떻게 채점하나	정확도 + f1(macro)

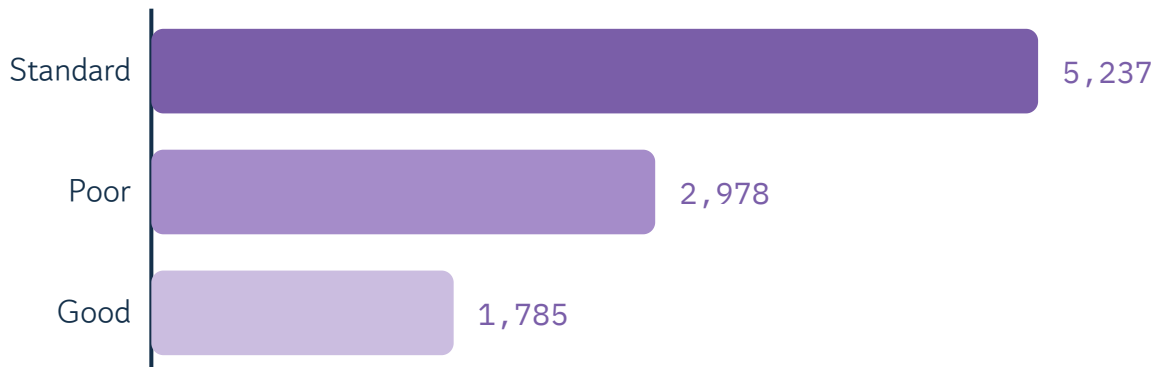
3번이 없어지니까 4번이 바뀝니다. 이게 이진분류와의 핵심 차이예요.

답이 2종이면 이진분류, 3종 이상이면 다중분류, 연속된 숫자면 회귀입니다. 이 한 줄로 문제 유형이 확정돼요.

```
train['Credit_Score'].unique()      # ['Poor' 'Standard' 'Good'] → 3종류니까 다중분류
```

Q3 등급마다 개수가 다르다

학습 데이터의 등급별 개수를 세어보면 이렇습니다.



가장 많은 등급이 가장 적은 등급의 약 3배

이렇게 개수가 쏠린 상태를 **불균형**이라고 합니다.

개수가 많은 등급은 모델이 자주 봐서 잘 맞히고, 적은 등급은 연습을 덜 해서 못 맞힙니다. 그런데 개수만 놓고 평균을 내면 이 사실이 숨겨져요. 다음 이야기가 그겁니다.

Q4 이 단원에서 가장 중요한 대목 — macro vs weighted

등급이 3개니까 f1 점수도 3개가 나옵니다. 이걸 **하나로 합쳐야** "이 모델 점수는 몇 점"이라고 말할 수 있죠. 합치는 방법이 두 가지입니다.

macro	weighted
등급마다 1표씩 (개수 무시)	개수만큼 표를 줌
Standard  1표	Standard  5,237표
Poor  1표	Poor  2,978표
Good (적고 어려움)  1표	Good  1,785표
약한 등급이 점수를 확 끌어내림 → 문제를 숨기지 않는다	많은 등급이 점수를 끌어올림 → 약한 등급이 묻힌다

같은 모델, 같은 예측인데 합치는 방법만 다릅니다.

실제 결과를 보면 이렇습니다.

```
f1(macro)      : 0.7107
f1(weighted)   : 0.7295 ← 더 높다
```

weighted가 더 높게 나옵니다. 개수가 제일 많은 Standard(f1 0.76)가 점수를 끌어올리고, 제일 적고 어려운 Good(f1 0.64)은 표가 적어서 묻히기 때문이에요.

두 값의 차이(약 0.02)가 곧 신호입니다.

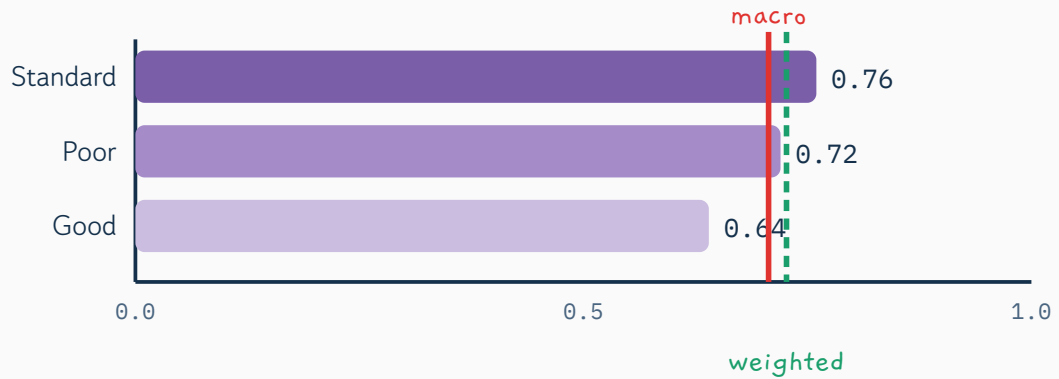
차이가 크다 = 소수 등급을 못 맞히고 있다는 뜻.

차이가 거의 없다 = 모든 등급을 고르게 맞히고 있다는 뜻.

Q5 직접 확인해보기

macro와 weighted, 누가 진실을 말하나

등급별 f1 점수를 직접 움직여보세요. **Good** 등급의 점수만 확 떨어뜨려도 weighted는 별로 안 움직입니다.



Standard f1 (5,237개) 0.76

Poor f1 (2,978개) 0.72

Good f1 (1,785개) - 여기를 움직여보세요 0.64

f1 macro	f1 weighted	차이
0.707	0.727	0.020

차이가 조금 있습니다. 개수 적은 등급이 살짝 약한 상태예요.

Good을 0.1까지 내려보세요. macro는 곤두박질치는데 weighted는 여전히 0.6 근처를 유지합니다. **weighted만** 보고 있었다면 "그럭저럭 괜찮네" 하고 넘어갔을 겁니다.

Q6 채점표는 3×3이 됩니다

이진분류의 혼동행렬이 2×2였다면, 등급이 3개면 3×3이 됩니다. **대각선이 맞힌 것**이고 나머지는 전부 틀린 거예요.

		모델의 예측 →		
		Poor	Standard	Good
→ 실제 등급	Poor	452	128	16
	Standard	143	821	84
	Good	11	129	216

← Good을 !

초록 대각선 = 맞힌 것 · 나머지 = 어떤 등급과 헷갈렸는지

이 표의 매력은 "어디서 헷갈리는지"가 보인다는 것. Good을 Standard로 착각하는 게 129건이네요.

Q7 코드로 보기

```
# ① 정답을 숫자로 (글자 그대로 뒤도 되지만, 등급처럼 순서가 있으면 숫자가 편하다)
y = train['Credit_Score'].map({'Poor':0, 'Standard':1, 'Good':2})

# ② 나눌 때 등급 비율 유지
X_tr, X_val, y_tr, y_val = train_test_split(
    train, y, test_size=0.2, random_state=0, stratify=y)

# ③ 학습
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=0)
rf.fit(X_tr, y_tr)
pred = rf.predict(X_val)

# ④ 채점 - average를 반드시 지정해야 한다
from sklearn.metrics import accuracy_score, f1_score, classification_report
print('정확도      : ', accuracy_score(y_val, pred))           # 0.7285
print('f1(macro)   : ', f1_score(y_val, pred, average='macro')) # 0.7107
print('f1(weighted): ', f1_score(y_val, pred, average='weighted')) # 0.7295
print(classification_report(y_val, pred)) # 등급별 점수를 한 번에 보여줌
```

`average` 를 안 쓰면 **에러가 납니다**. "f1이 3개 나왔는데 어떻게 합칠지 네가 정하라"는 뜻이에요.
기본은 **macro**로 두세요. 불균형을 숨기지 않는 정직한 점수입니다.

Q8 한 장 정리

	이진 분류	다중 분류
답의 개수	2개	3개 이상
양성(1)	있음 — 반드시 정해야 함	없음
ROC-AUC	쓸 수 있음	못 씀
f1	<code>f1_score(y, pred)</code>	<code>average='macro'</code> 필수
혼동행렬	2×2	3×3, 4×4 ...

이것만 기억하면 됩니다

- ① 답이 3종 이상이면 '양성'이라는 개념이 사라진다
- ② 그래서 f1을 합치는 방법(`average`)을 내가 정해야 한다
- ③ **macro**와 **weighted**의 차이가 곧 "소수 등급이 약하다"는 신호다

Q9 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 붓꽃 3품종 분류

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)          # 붓꽃 3품종(0,1,2)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
model = RandomForestClassifier(random_state=0).fit(X_tr, y_tr)
print('정확도:', round(accuracy_score(y_te, model.predict(X_te)), 3))
```

```
load_iris(return_X_y=True)
```

붓꽃 데이터(정답이 0/1/2 세 품종).

```
RandomForestClassifier(random_state=0).fit
```

랜덤포레스트로 학습(다중분류도 그대로).

```
accuracy_score(...)
```

세 품종을 얼마나 맞히나(정확도).

LEVEL 2 복잡 — 와인 3종 + 스케일링 + 클래스별 리포트

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X, y = load_wine(return_X_y=True)          # 와인 3종
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)

model = LogisticRegression(max_iter=5000).fit(X_tr_s, y_tr)
print(classification_report(y_te, model.predict(X_te_s)))  # macro/weighted 평균 확인
```

`load_wine(return_X_y=True)`

와인 3종(특성 13개, 단위 제각각).

`StandardScaler`

로지스틱은 크기에 민감 -> 같은 잣대로.

`classification_report`

각 종의 정밀도·재현율과 macro/weighted 평균을 봅니다(다중분류의 핵심).


```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv'
df = pd.read_csv(url).dropna()
X = df[['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']]
y = df['species'] # 3종: Adelie/Chinstrap/Gentoo

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
model = RandomForestClassifier(random_state=0).fit(X_tr, y_tr)
print(classification_report(y_te, model.predict(X_te)))
```

```
pd.read_csv(url).dropna()
```

펭귄 데이터를 URL에서 읽고 결측 행 제거.

```
y = df['species']
```

정답은 3종(Adelie/Chinstrap/Gentoo) 글자 그대로 써도 됩니다.

```
classification_report
```

종별 성적표로 어느 종이 잘/덜 맞는지 확인.